# Machine Learning

What machine learning is — Lecture 17 (Chapter 9, Part 1)

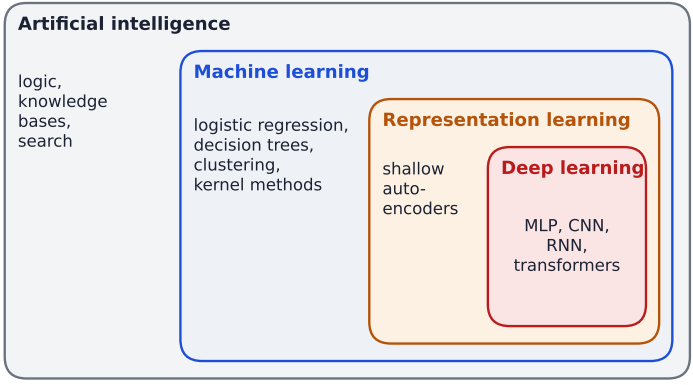
Amir Farbin

# Part 1 — What machine learning is

# From a test statistic to machine learning

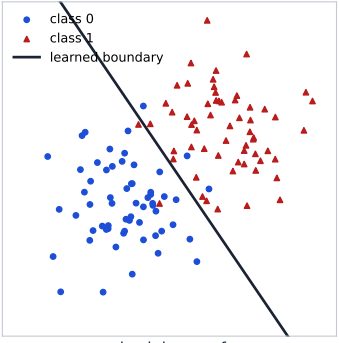
- The best test statistic is the **likelihood ratio** of signal to background
- In one dimension you can histogram it. In 18 you can't: 10 bins per feature is $10^{18}$ bins, almost all of them empty
- The way out: **project** the 18 numbers down to **one**, and cut on that
  - by hand: **feature engineering** ($M^2_{TR}$, invariant mass, ...)
  - learned from the data: **machine learning**

# AI, machine learning, deep learning

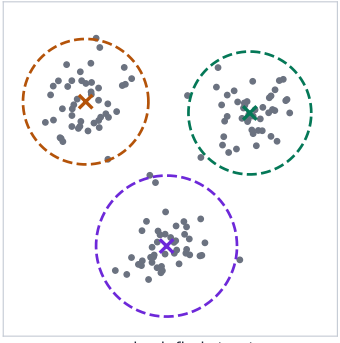



- **ML:** a generic algorithm, made specific by **data**

# Supervised and unsupervised




supervised: learn $y$ from $x$



unsupervised: find structure

- **Supervised:** the answer is known. Classification (cat or dog) or regression (the dog's weight)
- **Unsupervised:** no answer. Clustering, anomaly detection, PCA, k-means

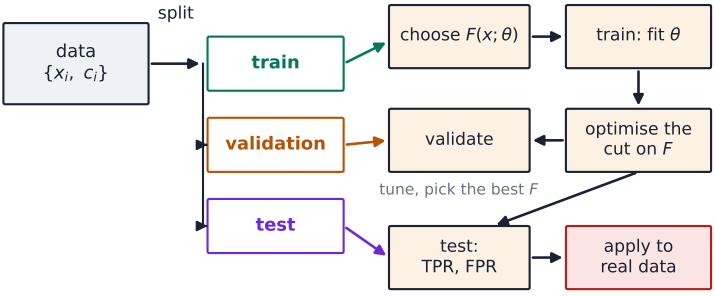
# Do the simplest thing first

- **Look at the data**, then try **one cut**, then a **linear model**, then something fancy
- A baseline tells you whether a fancy method failed or the problem is just hard
- Pick the fancy one when you can make a case for it

# The ingredients

- **Data:** examples of inputs, and for supervised learning the outputs you want
- **A model** $F(X \mid \theta)$: a function with dials, the parameters $\theta$
- **A cost:** one number that says how bad $F$ is on the data
  - supervised: compare $F$'s answer with the truth
  - unsupervised: score similarity, or reproduce the input through a bottleneck ($F(X) = X$, an **autoencoder**)
- **An optimizer:** turns the dials until the cost is small
- What training produces is **the parameters**

# The workflow for a signal search




The truth label $c$ is used to train and to test $F$ — never as an input. $F$ sees only $x$.

Real data has no $c$: that is what $F$ is for.

- $F$'s output is **a new feature**: histogram signal and background, pick the cut

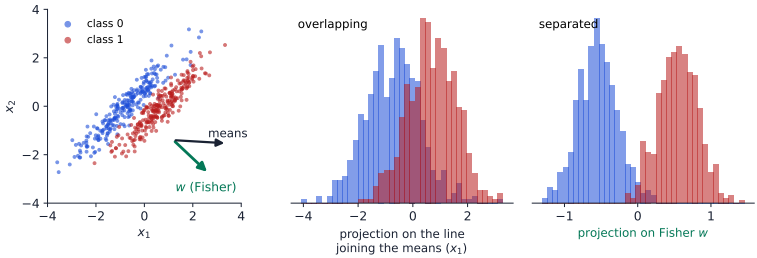
# Train, validation, test

- **Train:** fit $\theta$
- **Validation:** choose between models, settings and the cut
- **Test:** used **once**, at the end, with everything frozen
- Why three: a model can **memorize** its training data, and if you tune on a sample you fit its **fluctuations**

# Linear models

- $F(\vec{x}) = W\vec{x} + \vec{b}$: each output is a weighted sum of the inputs
- Find $W$ and $\vec{b}$ by minimizing the **mean squared error** $\frac{1}{N}\sum_i^N \left(F(\vec{x}_i \mid W, \vec{b}) - \vec{y}_i\right)^2$
- Simple example: $\vec{b} = 0$, $W$ is $1 \times d$. Stack the data: $X$ has the $N$ examples $\vec{x}_i$ as rows, $Y$ the $N$ answers
- **Analytic solution:** $W = (X^T X)^{-1} X^T Y$

# Fisher's linear discriminant (Lab 7)

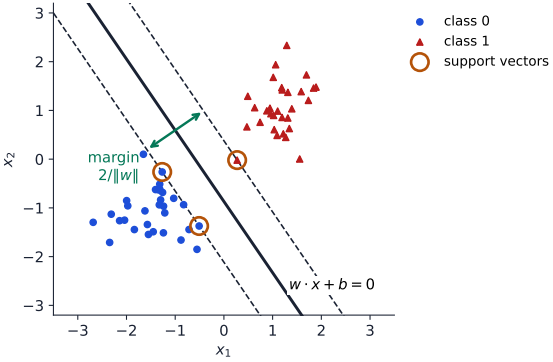



- Two classes with means $\vec{\mu}_0, \vec{\mu}_1$ and covariance matrices $\Sigma_0, \Sigma_1$; model $y = W\vec{x}$
- **Maximize** $S = \dfrac{(W \cdot \vec{\mu}_0 - W \cdot \vec{\mu}_1)^2}{W^T\Sigma_0 W + W^T\Sigma_1 W}$: means far apart, each class narrow
- **Analytic solution:** $W = (\Sigma_0 + \Sigma_1)^{-1}(\vec{\mu}_0 - \vec{\mu}_1)$

# Two losses, two analytic solutions

- **Mean squared error** $\rightarrow W = (X^T X)^{-1} X^T Y$: built from the **data** matrix $X$, $N \times d$
  - written in terms of the examples (the "dual" form, as the SVM uses next): $W = X^T (X X^T + \lambda I)^{-1} Y$, an inverse the **size of the data**, $N \times N$
- **Fisher's separation** $S \rightarrow (\Sigma_0 + \Sigma_1)^{-1}$: the **size of the features**, $d \times d$
  - $18 \times 18$ for SUSY, however many millions of events
- Almost nothing else in machine learning has a closed form, so in general we train **numerically**

# Every linear model draws a hyperplane

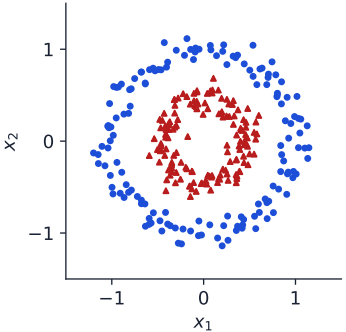



- **Support vector machine:** the **widest road** between the classes
- Only the points on the edge of the road matter: the **support vectors**

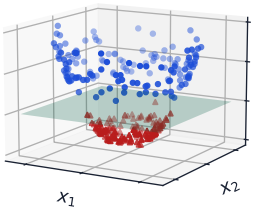
# The SVM as template matching

- $f(x) = \sum_i a_i \, y_i \, (x_i \cdot x) + b$, with $y_i = \pm 1$ and most $a_i = 0$
- Compare the new event with the stored signal $(+)$ and background $(-)$ examples
- The **kernel** replaces $x_i \cdot x$ by a similarity $k(x_i, x)$ measured through a nonlinearity

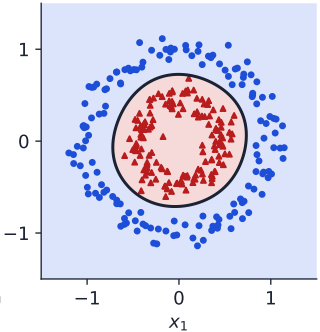
# Kernels straighten a curved boundary



no straight line separates them



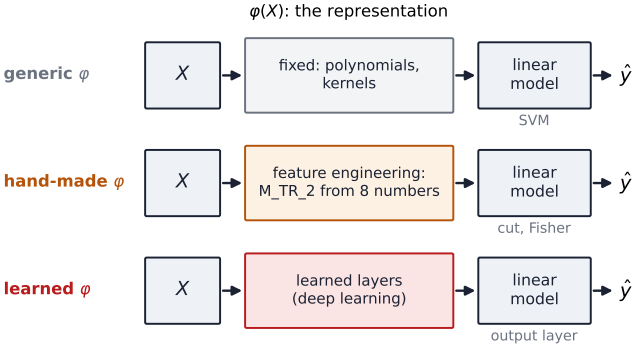
lifted: $\varphi(x) = (x_1, x_2, x_1^2 + x_2^2)$,
height $x_1^2 + x_2^2$: a plane separates them



RBF-kernel SVM boundary

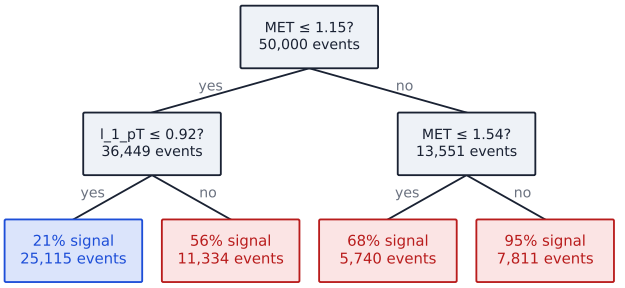
- Ring around a blob: no line in $(x_1, x_2)$, one cut in $x_1^2 + x_2^2$
- The trick: $\phi$ is never computed, only $k(x_i, x)$. Polynomial and Gaussian (RBF) kernels

# Features: generic, engineered, learned




- The detector's $10^8$ channels $\rightarrow$ reconstruction $\rightarrow$ **18 numbers**: SUSY is already engineered
- Every other method learns where to draw the boundary. **Deep learning learns the features**

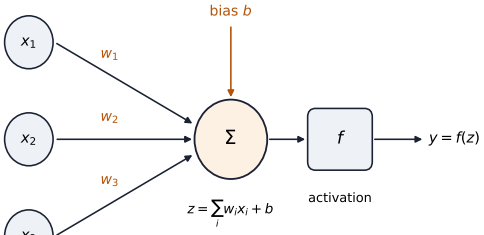
# Decision trees

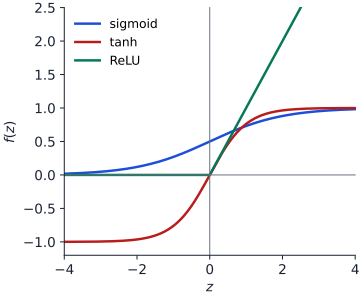



depth-2 tree offered M_TR_2, MET and l_1_pT; values in the dataset's own units

- Cut on one feature at a time until the leaves are pure. Not trained numerically: it's an algorithm
- **Random forest:** many trees **vote**. **Boosting:** each tree fixes the earlier ones' errors (XGBoost)

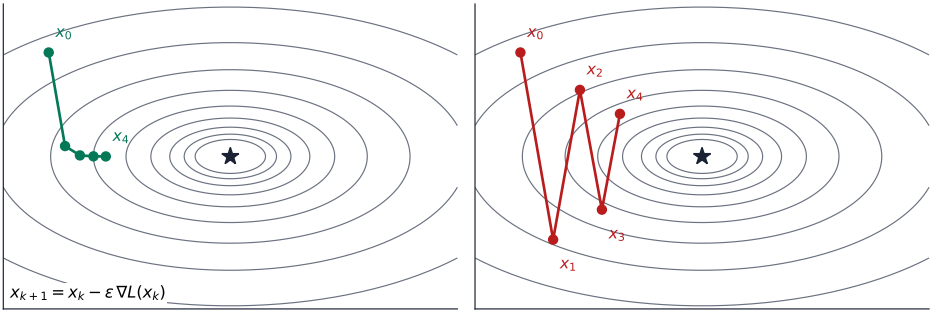
# A neuron




- $F(x) = f(Wx + b)$, with $f$ a **nonlinear** activation. If $f$ were linear, we'd be back at the linear model
- **Universal approximation (1989): one hidden layer** can approximate any continuous function, but nothing says how big it must be, or how to find it

# Training is walking downhill




learning rate $\varepsilon = 0.09$: small: slow but steady

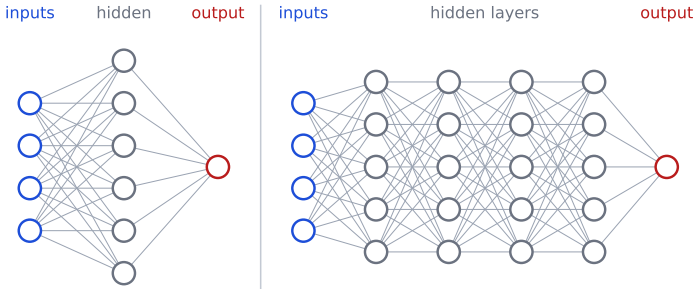
learning rate $\varepsilon = 0.18$: too large: zig-zags

- The cost $C(\theta)$ is a landscape over the parameters; start somewhere random
- **Gradient descent:** $\theta \leftarrow \theta - \epsilon \nabla C$, with $\epsilon$ the **learning rate**

# Two views of learning

- **Frequentist:** there is a best $\theta$; estimate it
- **Bayesian:** the data are fixed and $\theta$ is uncertain. Learn a **distribution** for each weight (a mean and a width) and get an **uncertainty** on the output
  - twice the parameters; ensembles of networks are the common alternative

# Shallow and deep networks




- The weights are the **wiring**. For SUSY: 18 inputs, hidden layers, 1 output
- Depth: one layer **builds** nonlinear features, the next **combines** them

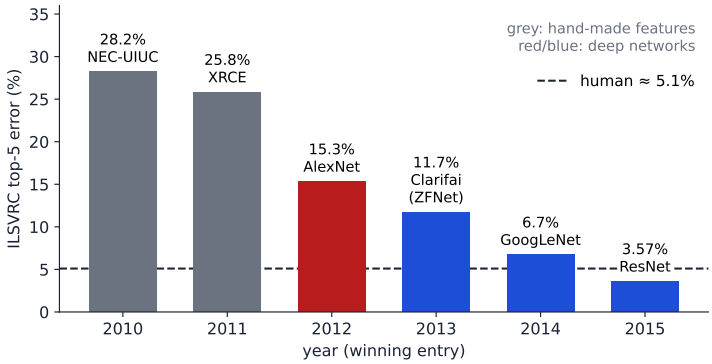
# A short history

- **1943** McCulloch and Pitts: a mathematical neuron
- **1958** Rosenblatt's perceptron, a machine with motor-driven resistors for weights
- **1965** first trained multi-layer networks; **1986** backpropagation popularized
- **1989** universal approximation; LeCun's network reads zip codes
- **1991** the **vanishing gradient**: in a deep network, early layers barely learn
- 1990s–2000s: SVMs, boosted trees and symbolic AI take over

# Why it works now

- **Activations:** ReLU's slope is 1 wherever it's on. Plus better initialization, layer-wise pre-training, normalization and residual connections
- **Data:** millions of labeled examples
- **GPUs:** built for games, they do exactly the arithmetic training needs
- **2014:** deep learning reaches particle physics, in the SUSY dataset's own paper

# 2012 and after




- Before 2012, vision ran on hand-made features. **AlexNet** (Toronto) changed that
- Atari from pixels and the score (2013–15); AlphaGo beats Lee Sedol (2016)

# Learning the features

- **Convolution:** slide a small filter over the image; it lights up where the pattern is
  - an identity-matrix filter is a **diagonal-line detector**
- Feature maps, pooling, repeat: **local and general** $\rightarrow$ **global and specific**
- The filters are **learned**: edges, then parts, then objects
- **Fine-tuning:** start from a network someone else trained and retrain the top layers